

Orion Content Platform - Phase 1 Validation Guide

Overview

This guide provides comprehensive validation procedures for testing all Phase 1 features across the three development streams: Core Platform, Security & Monitoring, and Quality Framework.

Health Check Endpoints

Primary Health Check

Endpoint: GET /api/health

Purpose: Overall system health verification

```
curl http://localhost:3000/api/health
```

Expected Response:

```
{
  "status": "healthy",
  "timestamp": "2025-09-13T12:00:00.000Z",
  "version": "1.0.0",
  "services": {
    "database": "connected",
    "redis": "connected",
    "auth": "operational",
    "apis": "operational"
  },
  "uptime": 3600,
  "environment": "development"
}
```

Database Connection Validation

Endpoint: GET /api/ops/status

Purpose: Database and core services status

```
curl http://localhost:3000/api/ops/status
```

Expected Response:

```
{
  "database": {
    "status": "connected",
    "latency": "12ms",
    "pool": {
      "active": 2,
      "idle": 8,
      "total": 10
    }
  },
  "redis": {
    "status": "connected",
    "latency": "3ms"
  },
  "external_apis": {
    "openai": "operational",
    "perplexity": "operational",
    "google": "operational"
  }
}
```

Security Monitoring Endpoints

Endpoint: GET /api/ops/metrics

Purpose: Security and performance metrics

```
curl -H "Authorization: Bearer <admin-token>" http://localhost:3000/api/ops/metrics
```

Expected Response:

```
{
  "security": {
    "failed_logins_24h": 0,
    "rate_limit_hits": 5,
    "active_sessions": 3
  },
  "performance": {
    "avg_response_time": "150ms",
    "requests_per_minute": 45,
    "error_rate": "0.1%"
  },
  "resources": {
    "memory_usage": "65%",
    "cpu_usage": "23%",
    "disk_usage": "45%"
  }
}
```

Quality Framework Validation

Endpoint: GET /api/ops/controls

Purpose: Quality framework status and metrics

```
curl -H "Authorization: Bearer <admin-token>" http://localhost:3000/api/ops/controls
```

Expected Response:

```
{
  "quality_gates": {
    "lighthouse_score": 85,
    "accessibility_score": 92,
    "seo_score": 88,
    "performance_score": 78
  },
  "automation": {
    "active_jobs": 2,
    "completed_today": 15,
    "failed_today": 0
  },
  "content_quality": {
    "avg_readability": 7.2,
    "seo_compliance": "94%",
    "duplicate_content": "0%"
  }
}
```

Authentication and Authorization Testing

1. Login Endpoint Validation

```
# Test admin login
curl -X POST http://localhost:3000/api/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "admin@orion-content.local",
    "password": "OrionAdmin2024!"
  }'
```

Expected Response:

```
{
  "success": true,
  "user": {
    "id": "user_123",
    "email": "admin@orion-content.local",
    "role": "ADMIN",
    "permissions": ["READ", "WRITE", "DELETE", "ADMIN"]
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires": "2025-09-14T12:00:00.000Z"
}
```

2. Role-Based Access Control (RBAC) Testing

Admin Access Test

```
curl -H "Authorization: Bearer <admin-token>" \
  http://localhost:3000/api/sites
```

Content Manager Access Test

```
curl -H "Authorization: Bearer <manager-token>" \
  http://localhost:3000/api/weeks
```

Reviewer Access Test

```
curl -H "Authorization: Bearer <reviewer-token>" \
  http://localhost:3000/api/weeks/1/approve
```

Unauthorized Access Test (Should Fail)

```
curl -H "Authorization: Bearer <viewer-token>" \
  -X DELETE http://localhost:3000/api/sites/1
```

Expected Response (403 Forbidden):

```
{
  "error": "Insufficient permissions",
  "required": "DELETE",
  "current": ["READ"]
}
```

Core Platform Feature Testing

1. Site Management

```
# Create new site
curl -X POST http://localhost:3000/api/sites \
  -H "Authorization: Bearer <admin-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Test Site",
    "url": "https://test-site.com",
    "type": "WORDPRESS",
    "credentials": {
      "username": "testuser",
      "password": "testpass"
    }
  }'
```

```
# List sites
curl -H "Authorization: Bearer <admin-token>" \
  http://localhost:3000/api/sites
```

```
# Get site details
curl -H "Authorization: Bearer <admin-token>" \
  http://localhost:3000/api/sites/1
```

2. Content Week Management

```
# Create content week
curl -X POST http://localhost:3000/api/weeks \
  -H "Authorization: Bearer <manager-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Week of September 13, 2025",
    "startDate": "2025-09-13",
    "endDate": "2025-09-19",
    "siteId": 1
  }'

# Get current week
curl -H "Authorization: Bearer <user-token>" \
  http://localhost:3000/api/weeks/current

# Approve week (reviewer only)
curl -X POST http://localhost:3000/api/weeks/1/approve \
  -H "Authorization: Bearer <reviewer-token>"
```

3. Daily Picks and Content

```
# Get daily picks
curl -H "Authorization: Bearer <user-token>" \
  http://localhost:3000/api/daily-picks

# Create topics for week
curl -X POST http://localhost:3000/api/weeks/1/topics \
  -H "Authorization: Bearer <manager-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "topics": [
      "AI in Content Marketing",
      "SEO Best Practices 2025",
      "Social Media Trends"
    ]
  }'
```

Security & Monitoring Validation

1. Rate Limiting Tests

```
# Test rate limiting (should succeed initially)
for i in {1..10}; do
  curl -w "%{http_code}\n" http://localhost:3000/api/health
done

# Test rate limiting (should return 429 after limit)
for i in {1..100}; do
  curl -w "%{http_code}\n" http://localhost:3000/api/health
done
```

2. Encryption and Credential Security

```
# Test credential storage
curl -X POST http://localhost:3000/api/credentials \
-H "Authorization: Bearer <admin-token>" \
-H "Content-Type: application/json" \
-d '{
  "service": "test-service",
  "credentials": {
    "api_key": "secret-key-123",
    "secret": "very-secret-value"
  }
}'

# Retrieve credentials (should be encrypted in storage)
curl -H "Authorization: Bearer <admin-token>" \
http://localhost:3000/api/credentials/test-service
```

3. Audit Logging Verification

```
# Check audit logs
curl -H "Authorization: Bearer <admin-token>" \
http://localhost:3000/api/ops/audit-logs?limit=10
```

Expected Response:

```
{
  "logs": [
    {
      "timestamp": "2025-09-13T12:00:00.000Z",
      "user": "admin@orion-content.local",
      "action": "LOGIN",
      "resource": "auth",
      "ip": "127.0.0.1",
      "success": true
    },
    {
      "timestamp": "2025-09-13T12:01:00.000Z",
      "user": "admin@orion-content.local",
      "action": "CREATE",
      "resource": "sites",
      "details": {"siteId": 1},
      "success": true
    }
  ]
}
```

Quality Framework Testing

1. Lighthouse Integration

```
# Trigger Lighthouse audit
curl -X POST http://localhost:3000/api/quality/lighthouse \
  -H "Authorization: Bearer <admin-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://test-site.com",
    "categories": ["performance", "accessibility", "seo"]
  }'
```

2. Content Quality Analysis

```
# Analyze content quality
curl -X POST http://localhost:3000/api/quality/analyze \
  -H "Authorization: Bearer <manager-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "content": "Sample content for quality analysis...",
    "checks": ["readability", "seo", "grammar"]
  }'
```

3. Automated Quality Gates

```
# Check quality gate status
curl -H "Authorization: Bearer <admin-token>" \
  http://localhost:3000/api/quality/gates/status
```

Integration Testing

1. WordPress Integration

```
# Test WordPress connection
curl -X POST http://localhost:3000/api/integrations/wordpress/test \
  -H "Authorization: Bearer <admin-token>" \
  -H "Content-Type: application/json" \
  -d '{
    "siteId": 1
  }'
```

2. Google Services Integration

```
# Test GA4 connection
curl -X POST http://localhost:3000/api/integrations/ga4/test \
  -H "Authorization: Bearer <admin-token>"

# Test Search Console connection
curl -X POST http://localhost:3000/api/integrations/gsc/test \
  -H "Authorization: Bearer <admin-token>"
```

3. 10Web Integration

```
# Test 10Web connection
curl -X POST http://localhost:3000/api/integrations/10web/test \
  -H "Authorization: Bearer <admin-token>"
```

Performance Testing

1. Load Testing Script

```
# Install Apache Bench (if not available)
sudo apt-get install apache2-utils

# Basic load test
ab -n 100 -c 10 http://localhost:3000/api/health

# Authenticated endpoint load test
ab -n 50 -c 5 -H "Authorization: Bearer <token>" \
  http://localhost:3000/api/sites
```

2. Database Performance

```
# Test database query performance
curl -H "Authorization: Bearer <admin-token>" \
  http://localhost:3000/api/ops/db-performance
```

Automated Testing

1. Run Integration Tests

```
npm test
```

2. Run Python Tests

```
cd python
source .venv/bin/activate
python -m pytest tests/ -v
```

3. Run Quality Framework Tests

```
npm run test:quality
```

Validation Checklist

Core Platform

- [] User authentication and authorization
- [] Site management (CRUD operations)
- [] Content week management

- ☐ Daily picks generation
- ☐ Multi-site support
- ☐ Role-based access control

Security & Monitoring

- ☐ Rate limiting functionality
- ☐ Credential encryption/decryption
- ☐ Audit logging
- ☐ Security headers
- ☐ Input validation
- ☐ SQL injection protection

Quality Framework

- ☐ Lighthouse integration
- ☐ Content quality analysis
- ☐ Automated quality gates
- ☐ Performance monitoring
- ☐ SEO compliance checking
- ☐ Accessibility validation

Integrations

- ☐ WordPress API connectivity
- ☐ Google Analytics 4 integration
- ☐ Google Search Console integration
- ☐ 10Web platform integration
- ☐ OpenAI API integration
- ☐ Perplexity API integration

Infrastructure

- ☐ Database connectivity
- ☐ Redis caching
- ☐ Environment configuration
- ☐ Error handling
- ☐ Logging system
- ☐ Health monitoring

Troubleshooting Common Issues

Authentication Failures

1. Check NEXTAUTH_SECRET is set
2. Verify database user table exists
3. Clear browser cookies
4. Check token expiration

Database Connection Issues

1. Verify DATABASE_URL format
2. Check database server status

3. Test connection with Prisma Studio
4. Verify user permissions

API Integration Failures

1. Verify API keys are correct
2. Check API quotas and limits
3. Test API endpoints independently
4. Review error logs

Performance Issues

1. Check Redis connection
2. Monitor database query performance
3. Review rate limiting settings
4. Analyze server resource usage

Success Criteria

Phase 1 is successfully validated when:

1. **All health check endpoints return healthy status**
2. **Authentication and RBAC work correctly for all user roles**
3. **Core platform features (sites, weeks, content) function properly**
4. **Security measures (rate limiting, encryption, audit logs) are active**
5. **Quality framework components are operational**
6. **All external integrations connect successfully**
7. **Performance meets acceptable thresholds**
8. **Automated tests pass with >95% success rate**

For detailed implementation information, see `PHASE1_IMPLEMENTATION_REPORT.md`.